

Moment Networks

June 8, 2026

Goal

What we want

For each Planck patch, use the data map d to learn the posterior

$$p(s \mid d).$$

We then want to use a NN to infer the first two moments:

$$\mathbb{E}[s \mid d], \quad \text{Std}[s \mid d].$$

Here

$$s = \begin{pmatrix} Q^s \\ U^s \end{pmatrix}, \quad d = \begin{pmatrix} Q^d \\ U^d \end{pmatrix}.$$

The output maps should have the original patch resolution, 384×384 .

Basic Supervised Problem

We train on synthetic signal/data pairs:

$$d_i = s'_i + n_i.$$

- s'_i : synthesized signal map from component-separated patch statistics.
- n_i : nuisance realization for the same patch.
- d_i : simulated observed data map.

The network learns a conditional map

$$d \longmapsto \text{posterior moments of } s \mid d.$$

Dataset Generation

For each patch p :

- 1 Start from the component-separated result

$$\tilde{s}_p = (\tilde{Q}_p, \tilde{U}_p).$$

- 2 Synthesize maps $s'_{p,i}$ matching scattering statistics.
- 3 Augment the synthesized maps.
- 4 Add same-patch nuisance realizations.

Current dataset scale:

$$10 \text{ syntheses per patch} \times 8 \text{ augmentations} = 80 \text{ pairs per patch.}$$

Files contain

$$(s'_Q, d_Q, s'_U, d_U)$$

for every generated pair.

Synthesis and Boundary Conditions

The real Planck patches are not periodic:

`pbk = False.`

For augmentation, we synthesize larger periodic maps and crop back:

Object	Shape	Boundary condition
Component-separated target	384×384	non-periodic
Running synthesis	larger map	periodic
Final training image	384×384	center crop

This avoids applying periodic translations directly to non-periodic observed patches.

Moment Network Loss

The reference implementations train a Gaussian moment network:

$$f_{\theta}(d) = (\mu_{\theta}(d), \log \sigma_{\theta}^2(d)) .$$

For training pairs $\{(d_i, s'_i)\}_{i=1}^N$, first learn the posterior mean:

$$\mathcal{L}_{\text{mean}} = \frac{1}{N} \sum_{i=1}^N \|s'_i - \mu_{\theta}(d_i)\|_2^2 .$$

Then learn mean and per-pixel marginal variance with the pixelwise Gaussian NLL:

$$\mathcal{L}_{\text{NLL}} = \frac{1}{2N|\Omega|} \sum_{i=1}^N \sum_{r \in \Omega} \left[\log \sigma_{\theta}^2(d_i)_r + \frac{(s'_{i,r} - \mu_{\theta}(d_i)_r)^2}{\sigma_{\theta}^2(d_i)_r} \right] .$$

Ω is the pixel grid. This is a diagonal Gaussian approximation, so it does not learn the full spatial covariance.

Why Did the Std Maps Collapse?

The mean is strongly supervised by channel-specific targets:

$$\mu_Q(d) \approx \mathbb{E}[s_Q \mid d], \quad \mu_U(d) \approx \mathbb{E}[s_U \mid d].$$

The std is learned indirectly through residual statistics:

$$\sigma^2(d)_r \text{ is driven by } (s'_r - \mu(d)_r)^2.$$

In the joint network, Q and U shared the same latent representation:

$$(Q^d, U^d) \mapsto (\log \sigma_Q^2, \log \sigma_U^2).$$

If the dominant residual patterns are common to Q and U, the variance heads can learn nearly the same uncertainty template.

Degrees of Freedom

During training, the degrees of freedom are the neural-network weights:

$$\theta.$$

During inference, the patch is fixed and the network produces maps:

$$\mu_{\theta}(d) \in \mathbb{R}^{384 \times 384}, \quad \sigma_{\theta}(d) \in \mathbb{R}^{384 \times 384}.$$

For polarization, this is done separately for Q and U .

Previous Version: Joint Network

The first implementation trained one shared Q/U network:

$$(Q^d, U^d) \mapsto (\mu_Q, \mu_U, \log \sigma_Q^2, \log \sigma_U^2).$$

This produced reasonable-looking posterior mean maps.

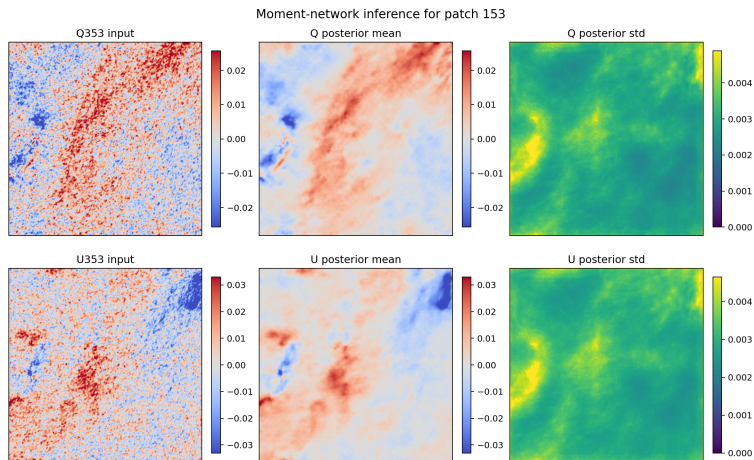
But the uncertainty maps were suspicious:

$$\sigma_Q(d) \approx \sigma_U(d)$$

in morphology.

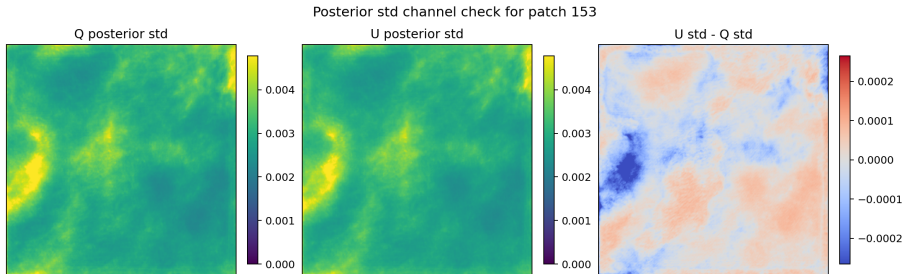
The likely issue was architectural: Q and U shared the full U-Net trunk.

Previous Version: Example Inference



Joint-network output for one patch. Mean maps look plausible, but Q/U uncertainty structure is too similar.

Previous Version: Std Diagnostic



The U -minus- Q std diagnostic exposed that the two std maps were close but not identical.

We can also do

$$Q^d \mapsto (\mu_Q, \log \sigma_Q^2),$$

$$U^d \mapsto (\mu_U, \log \sigma_U^2).$$

This avoids coupling the Q and U variance heads through a shared trunk.

Current Corrected Implementation

The corrected implementation trains two independent U-Nets:

Network	Input	Output
Q network	Q^d	$(\mu_Q, \log \sigma_Q^2)$
U network	U^d	$(\mu_U, \log \sigma_U^2)$

For a selected patch:

- 1 Load raw signal maps:

$$Q_{768}, U_{768} \in \mathbb{R}^{768 \times 768}.$$

- 2 Downgrade by 2×2 block averaging:

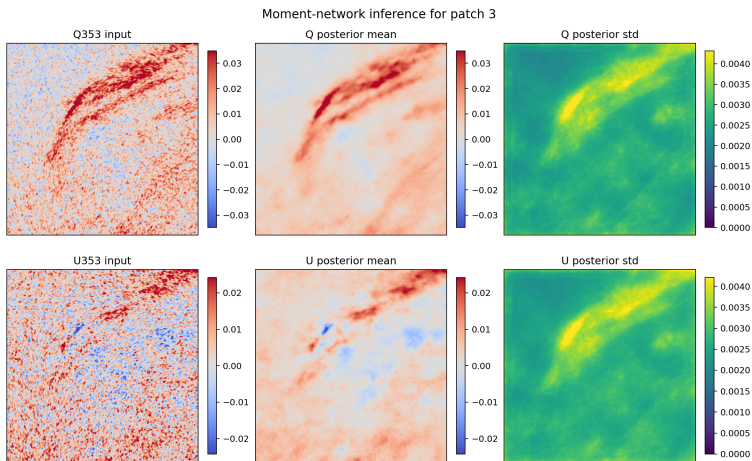
$$Q_{384}, U_{384} \in \mathbb{R}^{384 \times 384}.$$

- 3 Run Q and U networks independently.

Final products:

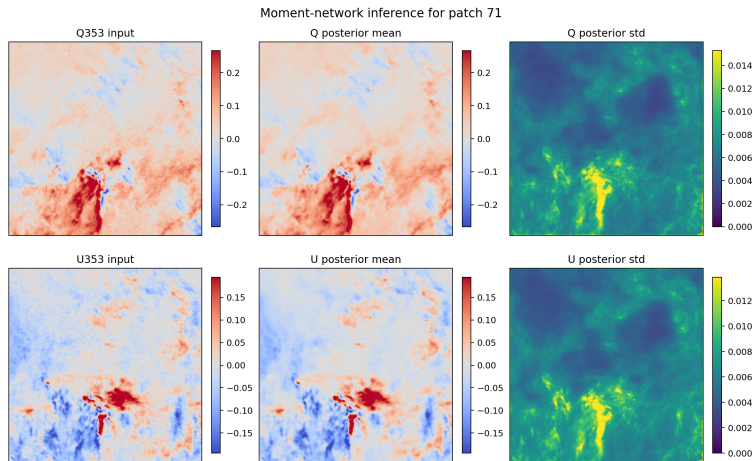
$$\mu_Q, \sigma_Q, \mu_U, \sigma_U.$$

Previous Version: More Examples



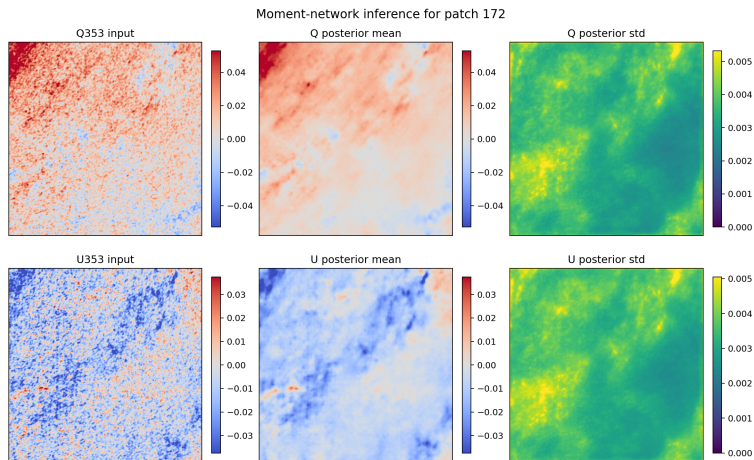
Example from the previous joint-network run.

Previous Version: More Examples



These plots are visual baselines for the corrected two-network run.

Previous Version: More Examples



After retraining, the same plotting script can compare independent Q/U uncertainty maps.

Current Status

- Dataset generation is complete for all patches.
- Previous joint-network results exposed a Q/U uncertainty coupling issue.
- Training has been corrected to two independent Q/U networks.
- Q and U training now runs in parallel on two GPUs.

Main check after retraining:

$$\sigma_Q(d) \quad \text{and} \quad \sigma_U(d)$$

should no longer share an artificial common morphology.